FRONTEND · JAVASCRIPT LIBRARY

React.js

Complete Syntax Reference & Notes

Core JSX, components, hooks, forms, routing, styling and everyday patterns — each with a short note and a working example.

Generated for personal reference · July 2026

Contents

01 Getting Started & Project Setup **4**

Vite (recommended) 4

Create React App (legacy) 4

Entry Point (main.jsx) 4

Folder Basics 4

02 JSX Fundamentals **5**

JSX Rules to Remember 5

03 Components **5**

Class Components (older syntax) 6

04 Props **6**

Default Props & children 6

05 State — useState **7**

Object & Array State 7

06 Event Handling **7**

07 Conditional Rendering **8**

08 Rendering Lists & Keys **8**

09 Forms — Controlled Inputs **9**

10 useEffect **9**

Fetching Data 10

11 useRef **10**

12 useContext **11**

13 useReducer **12**

14 useMemo & useCallback **12**

15 Custom Hooks **13**

16 Fragments **13**

17 Refs & forwardRef **14**

18 Error Boundaries **14**

19 Lazy Loading & Suspense **15**

20 React Router Basics **15**

21 Styling Approaches **16**

CSS Modules 16

22 Class Components & Lifecycle (Legacy Reference) **16**

23 Common Patterns & Best Practices **17**

24 Hooks Quick Reference **18**

01 Getting Started & Project Setup

Two common ways to bootstrap a React project today.

Vite (recommended)

```bash
npm create vite@latest my-app -- --template react
cd my-app
npm install
npm run dev
```

> **Note —** Vite gives near-instant server start and hot reload, and is the default choice for new projects.

Create React App (legacy)

```bash
npx create-react-app my-app
cd my-app
npm start
```

> **Note —** CRA is officially in maintenance mode — prefer Vite or a framework like Next.js for new apps.

Entry Point (main.jsx)

```jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

> **Note —** createRoot (React 18+) replaces the older ReactDOM.render API.

Folder Basics

- src/main.jsx or index.js — entry point, renders the App component
- src/App.jsx — the root component

- public/ — static assets served as-is
- package.json — scripts and dependencies

02 JSX Fundamentals

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It compiles down to React.createElement() calls.

```jsx
function Greeting() {
  const name = "Puru";
  return (
    <div className="greeting">
      <h1>Hello, {name}!</h1>
      <p>{2 + 2} is the answer</p>
    </div>
  );
}
```

Note — JSX must return a single root element — wrap siblings in a div or a Fragment.

JSX Rules to Remember

- Use className instead of class, and htmlFor instead of for
- Any valid JS expression can go inside curly braces { }
- Self-close empty tags: <img /> and <input />
- Multi-word attributes are camelCase: onClick, tabIndex

03 Components

Function components are the standard way to build UI in modern React.

```jsx
function Welcome() {
  return <h1>Welcome!</h1>;
}

// Arrow function style — equally common
const Welcome = () => <h1>Welcome!</h1>;

export default Welcome;
```

Class Components (older syntax)

Still found in legacy codebases. Function components with hooks have replaced this pattern for new code.

```jsx
import React, { Component } from 'react';

class Welcome extends Component {
  render() {
    return <h1>Welcome!</h1>;
  }
}
export default Welcome;
```

> **Note —** Naming matters — component names must start with a capital letter, or React treats them as plain HTML tags.

---

04 Props

Props pass data from a parent component into a child. They are read-only inside the child.

```jsx
function UserCard({ name, age }) {
  return <p>{name} is {age} years old</p>;
}

// Usage
<UserCard name="Puru" age={25} />
```

Default Props & children

```jsx
function Button({ label = "Click me" }) {
  return <button>{label}</button>;
}

function Card({ children }) {
  return <div className="card">{children}</div>;
}
// Usage: <Card><p>Content goes here</p></Card>
```

> **Note —** children is a special prop — whatever is placed between a component's opening and closing tags. For runtime type-checking, some older codebases use the prop-types package; most new codebases reach for TypeScript instead.

05 State — useState

useState adds local, re-render-triggering state to a function component.

```jsx
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+</button>
      <button onClick={() => setCount(c => c - 1)}>-</button>
    </div>
  );
}
```

Note — Prefer the updater-function form when the new state depends on the previous value — state updates are batched and asynchronous.

Object & Array State

```jsx
const [user, setUser] = useState({ name: '', age: 0 });
setUser(prev => ({ ...prev, name: 'Puru' }));

const [items, setItems] = useState([]);
setItems(prev => [...prev, newItem]);            // add
setItems(prev => prev.filter(i => i.id !== id)); // remove
```

Note — Always copy with spread — {...prev} or [...prev] — never mutate state directly.

06 Event Handling

React wraps native events in a SyntheticEvent, but the API feels just like the DOM.

```jsx
function Toolbar() {
  const handleClick = (e) => console.log('Clicked', e.target);
  const handleDelete = (id) => console.log('delete', id);

  return (
    <div>
      <button onClick={handleClick}>Click</button>
      <button onClick={() => handleDelete(5)}>Delete #5</button>
    </div>
  );
}
```

- onClick, onChange, onSubmit, onKeyDown, onMouseOver — all camelCase
- Pass a function reference, not a call: onClick={handleClick}, not onClick={handleClick()}

07 Conditional Rendering

```jsx
function Status({ isLoggedIn }) {
  if (isLoggedIn) return <p>Welcome back!</p>;
  return <p>Please log in.</p>;
}

// Ternary — good for inline JSX
const view = isLoggedIn ? <Dashboard /> : <Login />;

// && — renders nothing when the condition is falsy
{unreadCount > 0 && <Badge count={unreadCount} />}
```

Note — Watch out for the && pattern when the left-hand value can be zero — React renders the literal 0 instead of nothing. Convert it to a real boolean first, for example with a greater-than comparison.

08 Rendering Lists & Keys

```jsx
function ItemList({ items }) {
  return (
    <ul>
      {items.map(item => (
        <li key={item.id}>{item.name}</li>
      ))}
    </ul>
  );
}
```

Note — key must be a stable, unique id — avoid using the array index if the list can reorder, filter, or grow.

09 Forms — Controlled Inputs

```jsx
function LoginForm() {
  const [form, setForm] = useState({ email: '', password: '' });

  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm(prev => ({ ...prev, [name]: value }));
  };

  const handleSubmit = (e) => {
    e.preventDefault();
    console.log(form);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input name="email" value={form.email}
             onChange={handleChange} />
      <input name="password" type="password"
             value={form.password} onChange={handleChange} />
      <button type="submit">Login</button>
    </form>
  );
}
```

Note — A controlled input's value always comes from state — the single source of truth stays in React, not the DOM.

10 useEffect

```jsx
import { useEffect, useState } from 'react';

function Timer() {
  const [seconds, setSeconds] = useState(0);

  useEffect(() => {
    const id = setInterval(() => setSeconds(s => s + 1), 1000);
    return () => clearInterval(id); // cleanup
  }, []); // empty array = run once on mount

  return <p>{seconds}s elapsed</p>;
}
```

- No dependency array → runs after every render
- An empty array [] → runs once, after the first render
- [a, b] → re-runs whenever a or b changes
- The returned function cleans up before the next run, or on unmount

Fetching Data

```jsx
useEffect(() => {
  let ignore = false;
  fetch('/api/user')
    .then(res => res.json())
    .then(data => { if (!ignore) setUser(data); });
  return () => { ignore = true; };
}, []);
```

> **Note —** The ignore flag guards against setting state from a stale request after the component has re-rendered or unmounted.

11 useRef

```jsx
import { useRef } from 'react';

function TextInput() {
  const inputRef = useRef(null);
  const focus = () => inputRef.current.focus();

  return (
    <>
      <input ref={inputRef} />
      <button onClick={focus}>Focus</button>
    </>
  );
}
```

Note — Updating a ref does not trigger a re-render — use it for DOM handles or any mutable value that shouldn't drive rendering.

12 useContext

```jsx
import { createContext, useContext, useState } from 'react';

const ThemeContext = createContext();

function App() {
  const [theme, setTheme] = useState('dark');
  return (
    <ThemeContext.Provider value={{ theme, setTheme }}>
      <Toolbar />
    </ThemeContext.Provider>
  );
}

function Toolbar() {
  const { theme, setTheme } = useContext(ThemeContext);
  return (
    <button onClick={() => setTheme('light')}>
      Theme: {theme}
    </button>
  );
}
```

Note — Context avoids prop drilling — passing a value down through many layers that don't otherwise need it.

13 useReducer

```jsx
import { useReducer } from 'react';

function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'decrement': return { count: state.count - 1 };
    default: throw new Error('Unknown action: ' + action.type);
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, { count: 0 });
  return (
    <div>
      <p>{state.count}</p>
      <button onClick={() => dispatch({ type: 'increment' })}>+</button>
      <button onClick={() => dispatch({ type: 'decrement' })}>-</button>
    </div>
  );
}
```

Note — Reach for useReducer when state logic gets complex — many sub-values, or a next state that depends heavily on the action type.

14 useMemo & useCallback

```jsx
import { useMemo, useCallback, useState } from 'react';

function ExpensiveList({ items }) {
  const [count, setCount] = useState(0);

  const sorted = useMemo(
    () => [...items].sort((a, b) => a.value - b.value),
    [items]
  );

  const handleClick = useCallback(() => {
    console.log('clicked');
  }, []);

  return <List data={sorted} onClick={handleClick} />;
}
```

Note — Both memoize between renders — useMemo caches a value, useCallback caches a function reference. Reach for them to avoid expensive recalculation or needless child re-renders, not by default on every value.

15 Custom Hooks

```jsx
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);
    fetch(url).then(res => res.json()).then(d => {
      setData(d);
      setLoading(false);
    });
  }, [url]);

  return { data, loading };
}

// Usage
function Profile() {
  const { data, loading } = useFetch('/api/profile');
  if (loading) return <p>Loading...</p>;
  return <p>{data.name}</p>;
}
```

Note — A custom hook is just a function whose name starts with use and that calls other hooks — a clean way to reuse stateful logic.

16 Fragments

```jsx
function List() {
  return (
    <>
      <li>Item 1</li>
      <li>Item 2</li>
    </>
  );
}
// Equivalent to wrapping children in React.Fragment
```

Note — Use the long-form React.Fragment when the fragment needs a key prop — the shorthand syntax can't take one.

17 Refs & forwardRef

```jsx
import { forwardRef } from 'react';

const FancyInput = forwardRef((props, ref) => (
  <input className="fancy" ref={ref} {...props} />
));

// Parent:
// const ref = useRef();
// <FancyInput ref={ref} />
```

Note — forwardRef lets a parent attach a ref directly to a DOM node that's rendered inside a child component.

18 Error Boundaries

```jsx
class ErrorBoundary extends React.Component {
  state = { hasError: false };

  static getDerivedStateFromError() {
    return { hasError: true };
  }
  componentDidCatch(error, info) {
    console.error(error, info);
  }
  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }
    return this.props.children;
  }
}
```

Note — Error boundaries must be class components — there is no hook equivalent yet. Wrap risky subtrees with one.

19 Lazy Loading & Suspense

```jsx
import { lazy, Suspense } from 'react';

const Profile = lazy(() => import('./Profile'));

function App() {
  return (
    <Suspense fallback={<p>Loading...</p>}>
      <Profile />
    </Suspense>
  );
}
```

> **Note —** lazy and Suspense code-split a component so its bundle only loads when it's actually rendered.

20 React Router Basics

```jsx
import {
  BrowserRouter, Routes, Route,
  Link, useNavigate, useParams
} from 'react-router-dom';

function App() {
  return (
    <BrowserRouter>
      <Link to="/about">About</Link>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/user/:id" element={<UserProfile />} />
      </Routes>
    </BrowserRouter>
  );
}

function UserProfile() {
  const { id } = useParams();
  const navigate = useNavigate();
  return (
    <button onClick={() => navigate('/')}>
      Back home (user {id})
    </button>
  );
}
```

> **Note —** react-router-dom is the most widely used routing library for client-side React apps; install it separately with npm.

21 Styling Approaches

```jsx
// Inline style — a plain object, camelCase keys
<div style={{ color: 'red', fontSize: 20 }}>Text</div>

// Plain CSS file
import './App.css';
<div className="container">Text</div>
```

CSS Modules

```css
/* App.module.css */
.container {
  padding: 16px;
  border-radius: 8px;
}
```

```jsx
// Class names are scoped automatically
import styles from './App.module.css';
<div className={styles.container}>Text</div>
```

> **Note —** CSS Modules generate unique class names at build time, so styles never leak between components.

22 Class Components & Lifecycle (Legacy Reference)

```jsx
class Profile extends React.Component {
  componentDidMount() {
    // after the first render — fetch data, subscribe, etc.
  }
  componentDidUpdate(prevProps, prevState) {
    // after every re-render except the first
  }
  componentWillUnmount() {
    // cleanup — matches the function returned from useEffect
  }
  render() {
    return <div>{this.props.name}</div>;
  }
}
```

- componentDidMount corresponds to useEffect with an empty dependency array
- componentDidUpdate corresponds to useEffect with a dependency array
- componentWillUnmount corresponds to the cleanup function returned from useEffect

23 Common Patterns & Best Practices

- Lifting state up — move shared state to the closest common ancestor of the components that need it
- Composition over inheritance — build with children and props, not component subclassing
- Keep components small and focused — one clear responsibility each
- Colocate state — hold state as close as possible to where it is actually used
- Derive, don't duplicate — compute values from existing state or props instead of storing a copy

24 Hooks Quick Reference

A one-page summary of the hooks covered in this document.

Hook	Purpose	Returns
`useState`	Local component state	[value, setValue]
`useEffect`	Side effects — fetch, subscriptions, timers	undefined (or a cleanup fn)
`useRef`	Mutable value / DOM handle, no re-render	{ current }
`useContext`	Read a Context value	the context value
`useReducer`	Complex state logic via actions	[state, dispatch]
`useMemo`	Cache a computed value	the cached value
`useCallback`	Cache a function reference	the cached function
`useLayoutEffect`	Like useEffect, fires before paint	undefined